

Pemrograman Dasar TLS

Problem 1: TheLastAstronaut

DEKLARASI VARIABEL:

N, K, i, j, sisa_astronot, indeks_sekarang, nomor_eliminasinya : INTEGER

astronot : ARRAY OF INTEGER (Dinamis)

ALGORITMA:

BACA N (Jumlah total astronot)

BACA K (Nilai lompatan perhitungan awal)

ALOKASI memori untuk array astronot SEBANYAK N ELEMEN

UNTUK i DARI 0 HINGGA N - 1 LAKUKAN:

astronot[i] = i + 1

AKHIR UNTUK

sisa_astronot = N

indeks_sekarang = 0

CETAK "Urutan eliminasi: "

SELAMA sisa_astronot > 1 LAKUKAN:

indeks_sekarang = (indeks_sekarang + K - 1) MODULO sisa_astronot

nomor_eliminasinya = astronot[indeks_sekarang]

CETAK nomor_eliminasinya

JIKA (nomor_eliminasinya MODULO 2) SAMA DENGAN 0 MAKA:

K = K + 2

SEBALIKNYA:

K = K - 1

AKHIR JIKA

JIKA K < 2 MAKA:

K = 2

AKHIR JIKA

UNTUK j DARI indeks_sekarang HINGGA sisa_astronot - 2 LAKUKAN:

astronot[j] = astronot[j + 1]

AKHIR UNTUK

sisa_astronot = sisa_astronot - 1

AKHIR SELAMA

CETAK "Astronot terakhir yang bertahan: "

CETAK astronot[0]

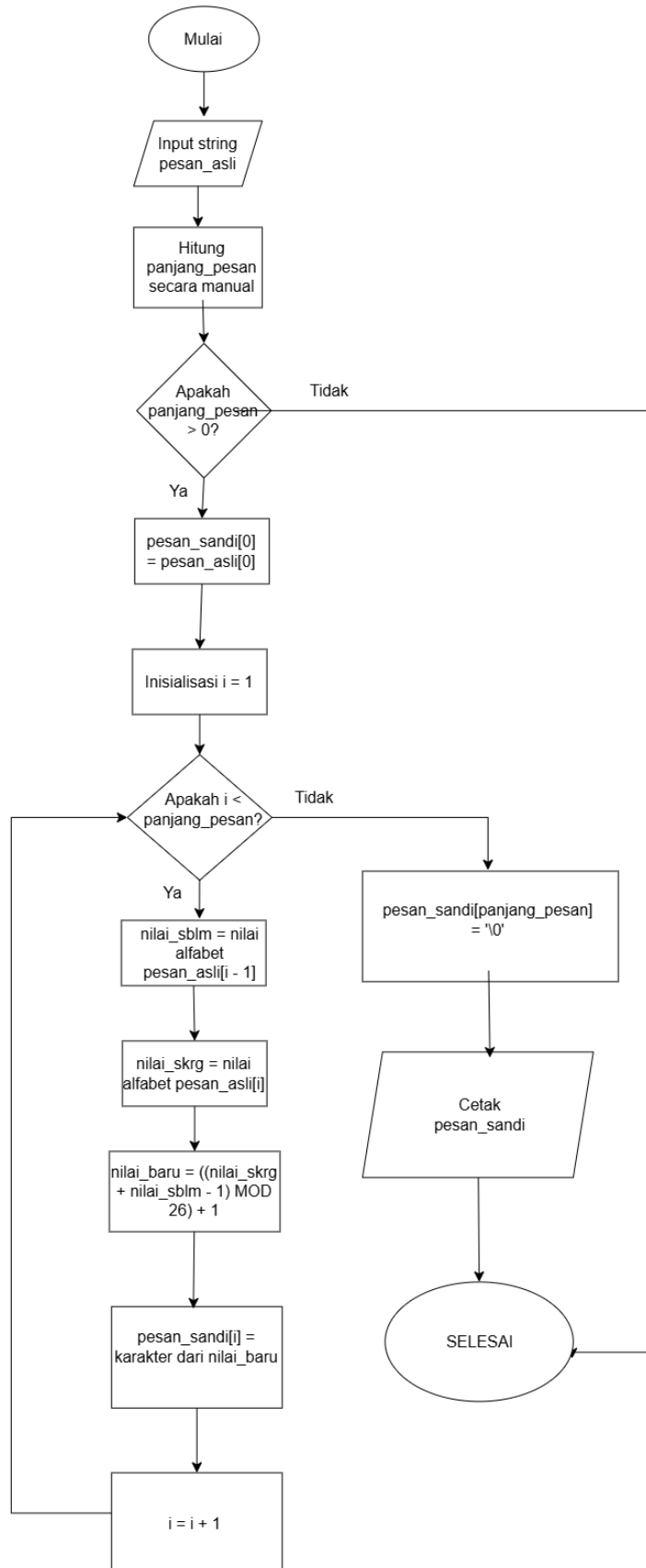
DEALOKASI memori array astronot

AKHIR ALGORITMA

Pendekatan penyelesaian untuk masalah "The Last Astronaut" menggunakan representasi *array* dinamis satu dimensi untuk menyimulasikan para astronot yang berdiri membentuk formasi lingkaran. Karena diwajibkan membangun algoritma secara manual tanpa fungsi dari *library* tingkat lanjut seperti *vector*, program menginisialisasi sebuah *array* berisi deret angka 1 hingga N. Untuk menghitung astronot yang terkena eliminasi pada hitungan ke-K secara efisien, algoritma menggunakan formula penentuan indeks berbasis sisa bagi (modulo), yaitu $(\text{indeks_sekarang} + K - 1) \% \text{sisa_astronot}$. Penggunaan fungsi modulo ini sangat krusial karena memastikan bahwa saat proses berhitung melampaui akhir barisan, hitungan akan otomatis memutar kembali ke elemen pertama, secara akurat merepresentasikan bentuk lingkaran yang terus menyambung tanpa menimbulkan *error* kelebihan batas indeks (*out of bounds*).

Setelah seorang astronot terpilih pada indeks tertentu, program mencatat nomor astronot tersebut yang tereliminasi, kemudian mengevaluasi aturan perubahan nilai K untuk eliminasi selanjutnya. Sesuai aturan, jika nomor astronot yang tereliminasi bernilai genap, maka nilai K ditambahkan 2, jika ganjil, nilai K dikurangi 1, dengan syarat validasi bahwa batas paling minimum nilai K adalah 2. Langkah berikutnya adalah menghapus astronot tersebut dari lingkaran. Hal ini dapat dilakukan dengan metode *shifting* atau penggeseran letak memori: seluruh sisa elemen astronot yang berada di sebelah kanan dari astronot yang gugur digeser satu langkah ke sebelah kiri untuk menimpa celah kosong ($\text{astronot}[i] = \text{astronot}[i+1]$). Setelah digeser, batasan populasi ukuran lingkaran (*sisa_astronot*) dikurangi satu. Perulangan (hitung, eliminasi, dan geser) ini terus berjalan memutar selama populasi masih lebih dari satu, hingga akhirnya menyisakan elemen tunggal di dalam array sebagai astronot terakhir.

Problem 2: Alien-In-The-Middle



Logika penyelesaian untuk masalah problem 2 "Alien-In-The-Middle" ini berfokus pada pengolahan *string* dengan memanipulasi nilai desimal ASCII dari alfabet. Tantangan pertama pada penyelesaian ini adalah tidak diperbolehkannya menggunakan *function* bawaan seperti *strlen()* dari *library* luar (hanya boleh *iostream*). Oleh karena itu, langkah pertama program adalah menjalankan fungsi penghitung panjang karakter secara manual (*user-defined function*) yang melakukan iterasi hingga menemukan karakter penutup baris (`\0` atau *null terminator*). Setelah panjang *string* diketahui, alokasi *array* sandi disiapkan. Program kemudian langsung menyalin karakter indeks ke-0 (huruf pertama pesan asli) secara utuh ke *array* hasil sandi, karena aturan menetapkan bahwa huruf pertama tidak akan mengalami proses pergeseran sandi apa pun.

Untuk proses pergeseran dari huruf kedua hingga huruf terakhir, digunakan sebuah struktur perulangan (*loop*). Pada tiap iterasinya, program mengambil karakter pesan asli yang sedang ditinjau dan mengonversinya menjadi representasi nilai alfabetis berstandar angka, dari rentang 1 hingga 26 (di mana A=1, B=2, dst). Nilai ini dikalkulasikan dengan cara mengurangi nilai ASCII karakter tersebut dengan ASCII 'A', lalu ditambah satu. Program melakukan konversi serupa pada karakter pesan asli yang posisinya persis sebelum huruf yang ditinjau. Kedua angka numerik ini kemudian dijumlahkan untuk mendapatkan letak huruf sandi yang baru. Agar pergeseran huruf yang melewati batas 'Z' bisa kembali berputar secara siklik ke awal alfabet (kembali ke 'A'), program membungkus total penjumlahan tersebut menggunakan operasi matematika berbasis modulo 26. Angka hasil pergeseran lalu dikonversi kembali menjadi tipe data karakter untuk melengkapi susunan *string* pesan rahasia sang alien.